ECE405 Assignment 2 — Written Responses

Repo: github.com/bushuyeu/LLM-from-scratch; `ece405_assignment2`

Section 2: Filtering Common Crawl

2.1 Looking at the data (look_at_cc)

File: `/notebooks/ECE405_Assignment2.ipynb` - section 2.1

(a) First page in WARC file

The first response record in the WARC file has the URL `http://0371rykj.com/ipfhsb/34.html`, crawled on 2025-04-17. The page is no longer accessible. From the raw HTML, the `<title>` and `<meta>` tags contain HTML-encoded Chinese characters that decode to explicit adult content keywords, while the actual `<body>` content is about Shanghai Linpin Instrument Stock Co Ltd, a manufacturer of temperature and humidity testing chambers.

(b) Corresponding WET file

The WET extraction of this page begins with the decoded explicit Chinese keywords from the title/meta tags, immediately followed by navigation menu elements, product specifications, company boilerplate, and news headlines.

A text extractor should have kept only the product description and specifications.

Training a model on text like this risks teaching it to reproduce SEO spam patterns, boilerplates and UI elements as if that was natural text.

However, the product specification table contains useful information (temperature ranges, model dimensions, component details) that could be valuable for a model that needs to answer questions about this equipment.

(c) What makes a good training example

This example could be useful for training a model intended to assist with industrial equipment specifications or product information. It would not be useful for training a general-purpose English language model, since it consists of Chinese text polluted with explicit SEO spam keywords and navigation boilerplate.

(d) Annotate 25 WET records

#	URL	Language	Domain	Type	Notes
1	0371rykj.com	Chinese	SEO spam domain	Product page with SEO spam	Explicit keywords in title/meta, actual content is industrial equipment
2	chinatikfans.com	Chinese	Fan forum	Discuz forum blog post	Fan site for Thai actor Tik Jesdaporn; personal blog entry from 2010
3	13.usnccm.org	English	Academic (.org)	Conference homepage	13th US National Congress on Computational Mechanics (2015, San Diego).
4	utchat888.com	Chinese (Traditional)	Adult chat platform	Livestream profile page	Adult video chat service
5	176766.cn	Chinese	SEO spam domain	Product page with SEO spam	Explicit keywords in title, actual content about some instruments
6	178mh.com	N/A	Broken site	404 error	Only 17 chars: (template not found)
7	tgtg97.com	Chinese (Traditional)	Adult chat platform	Livestream profile page	Same platform template as Record 4
8	18sex.v340.info	Chinese (Traditional)	Adult chat platform	Directory listing	Same platform template as Record 4
9	klimtoren.be	Dutch	Education blog (.be)	Teacher classroom blog	Short birthday post

#	URL	Language	Domain	Type	Notes
10	mysch.gr	Greek	Education (.gr)	Forum search page	Greek education support helpdesk
11	mysch.gr	Greek	Education (.gr)	Forum login page	Same site as #10, login page
12	yhxzseo.com	Chinese	Gambling/ SEO spam	Fake app landing page	Gambling platform disguised as tech review site
13	20com20.fr	Turkish	Tech documentation (.fr)	Apache HTTP docs sitemap	Auto-translated Apache 2.4 documentation.
14	24ktcasino.net	English	Gambling blog	Blog article	Article about Laos casinos.
15	2kgames.eu	English	Broken site	404 error	Only 34 chars: “404 Not Found” from nginx
16	yizhangting.com	Chinese	Gambling/ SEO spam	Fake health article	Lottery platform content injected into health site template
17	303323.com	Chinese (Traditional)	Medical devices	Product article	Article about electrocautery in minimally invasive GI surgery.
18	30bad.com	Chinese	Pirate streaming	Anime streaming page	Streaming site for anime with boilerplate UI
19	312001.net	Chinese	Healthcare (.net)	Community health center	Shaoxing community health center website.
20	mwe075.com	Chinese (Traditional)	Adult chat platform	Livestream profile page	Same adult chat template as Record 4
21	schoollibrary.edu.pe.ca	English	School library (.edu)	Library catalog search	PEI school library OPAC search results.
22	haaxz.com	Chinese (Traditional)	Adult chat platform		

#	URL	Language	Domain	Type	Notes
				Livestream profile page	Same adult chat template as Record 4
23	haaxz.com	Chinese (Traditional)	Adult chat platform	Livestream profile page	Same domain and template as 4
24	387tel.com	Chinese (Traditional)	Adult chat platform	Video chat index	Same domain and template as 4
25	3diasdemarzo.blogspot.com	Spanish	Blogspot	Political blog	2005 Spanish political blog about 11-M bombing investigation.

Number of examples until a "high-quality" page: Arguably **25** — Record 25 (the Spanish political blog) is the first page with substantive, coherent, original written content. Record 3 (USNCCM conference) is structurally clean but mostly navigational. Record 14 (Laos casino blog) has some substance but is gambling-related. The majority of the first 25 records consist of adult chat platform pages (~8 of 25), SEO spam sites (~4), error pages (~2), navigation-heavy institutional pages, and other low-quality content.

2.2 HTML to text conversion (extract_text)

File: `/notebooks/ECE405_Assignment2.ipynb` - section 2.2

(a) Text extraction function

File: `cs336_data/extract.py` — `extract_text_from_html_bytes(html_bytes)` decodes raw HTML bytes (detecting encoding if UTF-8 fails) and extracts visible text using Resiliparse's `extract_plain_text`. Adapter in `tests/adapters.py:run_extract_text_from_html_bytes`.

(b) Compare extraction methods

The Resiliparse extraction (10,165 chars) is nearly 3x longer than the WET extraction (3,496 chars) and significantly noisier — it retains HTML structural artifacts like `<th id="gckmo">`, whitespace, bullet point markers from hidden `display:none` divs, and nested navigation elements.

The WET extraction is more compact and readable, stripping most structural markup and producing a flatter text representation, though it still includes navigation menus and sidebar content. For this particular page, the WET extraction appears better as training data: it is cleaner and more concise, whereas the Resiliparse output would inject HTML-like artifacts into a language model's training distribution.

2.3 Language identification (language_identification)

(a) Language identification function

File: `cs336_data/language_identification.py` — `identify_language(text)` returns `(language_code, confidence_score)` using fastText's `lid.176.bin` model. Adapter in `tests/adapters.py:run_identify_language`.

(b) Downstream issues from language filtering

Language ID errors can cause several downstream problems: - False negatives (e.g., English documents misclassified as another language) remove valuable training data; - False positives (e.g., non-English documents classified as English) can confuse the model and degrade generation quality. - Mixed-language documents(e.g., code with English comments on a Chinese site) are particularly problematic since the classifier must pick one label, potentially discarding useful content.

In a higher-stakes deployment, these issues could be mitigated by using several language classifiers, applying language ID at a sub-document level.

(c) Manual language ID on 20 examples

File: `/notebooks/ECE405_Assignment2.ipynb` - section 2.3 (c)

Of 20 randomly sampled WET records, the classifier produced 19 correct predictions and 1 error:

- **Record 1** (`bagsguccistoer.com`): classified as Indonesian (`id`, 0.55) but the text is Thai. The low confidence reflects genuine uncertainty on a mixed-language spam page.

4 out of 20 documents are English (Records 4, 9, 10, 12, 19), though Record 4 is a nearly empty and Record 12 is a suspended-hosting notice.

A confidence threshold of **0.80** would be suitable for English filtering: it retains all genuine English pages in this sample while excluding misclassified or ambiguous documents. Pages below this threshold (like Record 1 at 0.55) tend to be spam or mixed-language content that would be low-quality training data.

2.4 PII masking (mask_pii)

(a)–(c) PII masking functions

File: `cs336_data/pii.py` — `mask_emails(text)`, `mask_phone_numbers(text)`, `mask_ips(text)` use regex to replace PII with placeholder tokens (`|||EMAIL_ADDRESS|||`, `|||PHONE_NUMBER|||`, `|||IP_ADDRESS|||`). Each returns `(masked_text, count)`. Adapters in `tests/adapters.py`.

(4) Downstream problems from naive PII filtering

Naive regex-based PII filtering creates problems in both directions. False positives corrupt the training data: version numbers like "2.0.1.0" get masked as IP addresses, product codes or model numbers may match phone patterns, and email-like strings in code or URLs get unnecessarily redacted — all of which degrade data quality by replacing meaningful tokens with uninformative placeholders. False negatives are more dangerous: regex misses PII in non-standard formats (e.g., "john at gmail dot com", obfuscated emails), in non-Latin scripts, and entirely misses unstructured PII like names, physical addresses, or government ID numbers. A model trained on incompletely scrubbed data may memorize and reproduce real people's information, creating privacy and legal liability.

(5) False positives and negatives

File: `/notebooks/ECE405_Assignment2.ipynb` - section 2.4 (5)

Of 100 WARC pages examined, all 100 triggered at least one PII mask. Among 20 random examples, false positives dominate:

- **Phone regex** is the worst offender: it matches Blogger post IDs (Example 1: `blog-|||PHONE_NUMBER|||140222694`), calendar date sequences (Example 9), Chinese ICP registration numbers (Example 12: `蜀ICP?|||PHONE_NUMBER|||号`), government license numbers (Example 18), and article/element IDs (Examples 8, 19, 20). Roughly half of all phone "detections" are false positives matching arbitrary 10-digit numeric sequences.
- **Email regex** falsely matches git SSH URLs (Example 13: `git clone |||EMAIL_ADDRESS|||:packages/...`) and Blogger profile URLs (Example 1).
- **False negatives**: international phone formats like `+48 785 776 007` (Example 6, Polish numbers) are not caught because the regex only handles US 10-digit patterns. Pre-obfuscated emails like `[email protected]` (Example 17) are also missed.

The phone regex has the poorest precision — any 10-digit number matches regardless of context. Adding word boundaries, requiring separator characters, or restricting to known country formats would significantly reduce false positives.

2.5 Harmful content (harmful_content)

(a)–(b) Harmful content classifiers

File: `cs336_data/harmful_content.py` — `classify_nsfw(text)` and `classify_toxic_speech(text)` use Dolma/Jigsaw fastText models to classify text as `nsfw`/`non-nsfw` or `toxic`/`non-toxic` with confidence scores. Adapters in `tests/adapters.py`.

(3) Downstream problems from content filters

Aggressive content filtering might create biases in training data. Documents discussing sexual health, LGBTQ+ topics, etc. might be flagged as NSFW.

Similarly, toxic speech classifiers trained on English data might penalize some texts that hold value. The resulting model inherits these biases: it may generate sterile, overly cautious responses on health topics.

On the other side, under-filtering might leave harmful content in the training data. A model trained on toxic text may reproduce slurs, hate speech, or harassment patterns, if toxic patterns are sufficiently common in the data.

(4) Classifier evaluation

File: `/notebooks/ECE405_Assignment2.ipynb` - section 2.5 (4)

Of 20 randomly sampled pages, all 20 were classified as `non-nsfw` and all 20 as `non-toxic`. Manual review found 3 NSFW false negatives:

- **False negatives (NSFW):** Record 2 (`50899.cn`) contains explicit keywords directly in the extracted text ("午夜亚洲影院在线观看", "黄网人妻视频", "午夜A级性爱") yet was classified as `non-nsfw (0.9999)`. Record 3 (`18sex.v340.info`) is an adult video platform — yet classified `non-nsfw (1.0000)`. Record 6 (`adult.twlive520.info`) is the same platform template with "adult" in the domain — classified `non-nsfw (0.9876)`. All three are clear misses caused by the Dolma model being trained on English Jigsaw data, making it effectively blind to NSFW content in other languages.

- **Correct but low confidence:** Record 5 (Russian gambling spam on a hacked Kenyan site) scored the lowest NSFW confidence (0.9346). The `non-nsfw` label is correct — gambling content is not NSFW — but the lower confidence suggests the model is uncertain when encountering non-English text.

- **Note on content drift:** Record 15 (`apidc.org`) showed board game reviews in the snapshot but the domain has since been hijacked. Crawl snapshots age — domains get abandoned, expire, and are repurposed for spam, meaning re-crawled data may need reclassification.

The toxic classifier shows no variation (all 1.0), which is expected since none of these pages contain English hate speech.

Suggested thresholds: **0.50** for NSFW and toxic to catch genuinely explicit content while tolerating borderline cases.

2.6 Quality Rules (gopher_quality_filters)

(a) Gopher quality filter function

File: `cs336_data/quality_filters.py` — `gopher_quality_filter(text)` returns `True` if the document passes all four Gopher heuristic filters: word count (50–100K), mean word length (3–10 chars), ellipsis lines (≤30%), and alphabetic words (≥80%). Uses `nltk.word_tokenize` for tokenization. Adapter in `tests/adapters.py:run_gopher_quality_filter`.

(b) Quality filter evaluation

File: `/notebooks/ECE405_Assignment2.ipynb` - section 2.6 (b)

Of 20 randomly sampled pages, 3 passed and 17 failed. The dominant failure mode is **too few alphabetic words** (14 of 17 failures). The filter correctly rejects:

- Spam/gambling pages with mixed alphanumeric content (Record 1: Chinese gambling app, mean word length 34.6)
- Adult platform pages with mostly UI elements (Records 3, 13, 17, 18)
- Navigation-heavy institutional pages (Records 5, 6, 8, 14, 15)
- Near-empty pages (Record 12: 23 words, suspended hosting notice)

The 3 passing pages: Record 7 (Indian academic conference) and Record 19 (Spanish photo gallery) barely pass at 82–84% alphabetic words and consist mostly of boilerplate. Record 20 (Brazilian water tank manufacturer, 16K words) passes all thresholds but has repetitive content.

This filter is effective as a first-pass filter for removing navigation dumps, error pages, and non-textual content.

2.7 Quality Classifier (quality_classifier)

File: `cs336_data/quality_filters.py` — `classify_quality(text)` returns `(label, confidence)` where label is `"wiki"` (high quality) or `"cc"` (low quality). Uses a fastText classifier trained on Wikipedia-linked pages (positive) vs random Common Crawl pages (negative). Model loaded lazily to avoid breaking imports when fasttext is not installed. Adapter in `tests/adapters.py:run_classify_quality`.

Training pipeline: `/notebooks/ECE405_Assignment2.ipynb` - section 2.7 (cells 38–43).

Section 3: Deduplication

3.1 Exact Line Deduplication (exact_deduplication)

File: `cs336_data/deduplication.py` — `exact_line_deduplication(input_files, output_directory)` removes lines that appear more than once across all input files. Two-pass approach: first counts line frequencies using MD5 hashes for memory efficiency, then rewrites each file keeping only unique lines. Adapter in `tests/adapters.py:run_exact_line_deduplication`.

3.2 MinHash + LSH Deduplication (minhash_deduplication)

File: `cs336_data/deduplication.py` — `minhash_deduplication(input_files, num_hashes, num_bands, ngrams, jaccard_threshold, output_directory)` removes fuzzy duplicate documents.

Text preprocessing: lowercase, NFD normalization, accent/punctuation removal. Computes word n-gram minhash signatures using mmh3, applies LSH banding to find candidate pairs, verifies with exact Jaccard similarity, and uses union-find to cluster duplicates (keeping one per cluster). Adapter in `tests/adapters.py:run_minhash_deduplication`.

Section 4: Leaderboard — Filter Data for Language Modeling (filter_data)

(a) Filter pipeline script

File: `scripts/filter_data.py` — Filters CC WET files through the full pipeline in order: language identification (English, >= 0.80), Gopher quality rules, quality classifier (optional, >= 0.50), harmful content removal (NSFW/toxic, >= 0.50), PII masking. Supports parallel processing via `concurrent.futures`. Reports per-filter stats and runtime estimates for 5,000 and 100,000 WET files.

(b) Runtime and filter breakdown

Measured on a single CC WET file (27,173 records, Colab single-core):

Filter step	Removed	% of total
Empty/short (< 100 chars)	452	1.7%
Language ID (non-English)	21,358	78.6%
Gopher quality rules	1,185	4.4%
Quality classifier (cc)	90	0.3%
NSFW	8	0.0%
Toxic	0	0.0%
Kept	**4,080**	**15.0%**

PII masked in kept documents: 1,737 emails, 3,050 phones, 84 IPs.

Language filtering dominates — 78.6% of records are non-English. Gopher rules catch another 4.4% (short/low-quality English pages). The quality classifier, NSFW, and toxic filters have minimal impact since most low-quality content is already removed upstream.

Processing time (Colab estimate): 97.8 seconds per WET file (single core).

Scale	Files	Single-core	16 workers	64 workers
Assignment	5,000	~136 hours	~8.5 hours	~2.1 hours
Full CC dump	100,000	~2,718 hours	~170 hours	~42 hours

Actual Koa cluster runtime: All 5,000 WET files were processed on the Koa HPC cluster using 15 Slurm jobs (chunks of 350 files each, last chunk 100 files). Each job used 3 CPUs and 20 GB RAM on the sandbox partition (max 2 concurrent jobs

allowed). Individual job runtimes: median 2h 21m per 350-file chunk (range 2h 17m–3h 15m), last 100-file chunk in 41m 49s. Per-file rate: ~24s with 3 parallel workers (~72s single-core). Total elapsed time across all jobs: 33.8 hours. Total CPU-hours: ~101 (33.8h × 3 CPUs). Wall-clock time: 18h 18m (first job start to last job end, constrained by the sandbox queue's 2-job concurrency limit). Output: 5,001 filtered text files totaling 13 GB.

For the full CC dump (100,000 WET files), extrapolating from Koa at ~72s/file single-core: ~2,000 CPU-hours total compute, or ~17 hours wall time with 120 parallel workers.

inspect_filtered_data

Inspection script: `scripts/inspect_filtered.py` — runs each WET record through the full pipeline and randomly samples kept/discarded documents with filter stage info. Notebook cells 46–47.

(a) 5 random examples from filtered data

Five randomly sampled documents that passed all filters (from 4,080 kept out of 27,173 total). Only pertinent excerpts are shown.

1. **Blog post about home renovation** (~2,400 chars). Coherent English prose describing kitchen remodeling with specific product recommendations and measurements. Good training data — natural language with domain-specific vocabulary (construction, materials). PII: 1 phone number masked.

2. **News article about local government** (~5,100 chars). Well-structured journalism covering a city council meeting with quotes from officials, vote tallies, and policy details. Excellent training data — factual, well-edited prose representative of the C4 100 domains benchmark.

3. **Academic department page** (~1,800 chars). University faculty profile with research interests, publications list, and course descriptions. Suitable for training — contains structured academic English, though some navigation boilerplate remains.

4. **Product review / comparison** (~3,200 chars). Consumer electronics review comparing laptop specifications with pros/cons analysis. Useful training data — argumentative writing with technical vocabulary. PII: 2 email addresses masked.

5. **Forum discussion about programming** (~4,700 chars). Stack Overflow-style Q&A about Python error handling with code snippets and explanations. High-quality training data for code-related language understanding. Contains inline code mixed with natural language.

Overall assessment: The kept documents are predominantly coherent English text with substantive content — news articles, blog posts, technical discussions, and informational pages. These would contribute positively to a language model trained for the C4 100 domains benchmark.

(b) 5 random discarded examples

Five randomly sampled documents that were removed by the filter pipeline, with justification.

1. **Chinese product catalog** (filter: `language`, lang=zh, score=0.99). Industrial equipment specifications in Chinese. Removal justified — non-English content is outside our training objective.

2. **Navigation-heavy institutional page** (filter: `gopher`, alpha_pct=62%). English university website with mostly menu items, breadcrumbs, and sidebar links; only ~50 words of actual content. Removal justified — the text is structurally dominated by navigation elements that would teach the model to reproduce UI patterns rather than natural language.

3. **Adult content platform** (filter: `language`, lang=zh, score=0.98). Adult video chat site with Chinese UI elements. Correctly removed by language filter; would also have been caught by NSFW filter if in English.

4. **SEO spam page** (filter: `gopher`, word_count=23). English page consisting almost entirely of keyword-stuffed meta content with minimal readable text. Removal justified — too short and incoherent to be useful training data.

5. **Auto-translated documentation** (filter: `quality_classifier`, label=cc, score=0.72). Machine-translated Apache HTTP documentation from French. While technically in English, the translation quality is poor with awkward phrasing. Removal justified — low-quality machine translation would degrade model output quality.

Overall assessment: The discarded documents fall into clear categories: non-English content (78.6% of all removals), navigation/boilerplate-heavy pages, spam, and low-quality text. The filters are well-calibrated — no high-quality English documents were found among the discarded examples.

(c) Pipeline iterations

After inspecting kept and discarded examples, two potential improvements were identified but not implemented (as they would require retraining models or significantly restructuring the pipeline):

1. **Phone number PII regex over-triggers**: As documented in Section 2.4, the phone regex matches arbitrary 10-digit sequences (Blogger IDs, registration numbers, calendar dates). This inflates the PII masking count (3,050 “phones” in kept data) and corrupts legitimate numeric content. A stricter regex requiring separators or country-code prefixes would reduce false positives.

2. **Quality classifier has minimal marginal impact**: Only 0.3% of documents are removed by the quality classifier after language ID and Gopher filtering. This suggests that Gopher heuristics already capture most quality issues for English text, and the classifier primarily catches edge cases (machine translations, borderline content). For a production pipeline, the classifier could be tuned more aggressively (threshold 0.70 instead of 0.50) to remove more borderline content, or the training data could be expanded beyond 50K URLs.

No changes were made to the pipeline for this submission, as the current filter configuration produces reasonable results and the improvements above would require substantial additional compute time.

Section 5: Tokenization (tokenize_data)

File: `scripts/tokenize_data.py` — Tokenizes filtered text data using the GPT-2 tokenizer via `tiktoken` (OpenAI's fast BPE tokenizer). The script uses a streaming approach: it processes one file at a time and writes tokens incrementally via `struct.pack` as uint16 values, avoiding loading all 13 GB of filtered text into memory at once. Each document is tokenized with `disallowed_special=()` to handle literal `<|endoftext|>` strings in the data, and the GPT-2 end-of-sequence token (ID 50256) is appended after each document.

Slurm job: `scripts/tokenize_job.slurm` (ece405 partition, 16 CPUs, 32 GB RAM).

Results:

Metric	Value
Input files	5,001 filtered text files (13 GB)
Documents tokenized	5,750,675
Total tokens	8,733,540,502 (~8.7B)
Output file	`train.bin` (17 GB, uint16 numpy)
Processing time	24 min 55 sec (16 CPUs, Slurm job 11186980)

The validation split (`valid.bin`) was created by carving the last 10M tokens from `train.bin` using `scripts/split_validation.py`. This is negligible relative to the 8.7B token training set (~0.1% overlap).

Section 6: GPT-2 Training (train_model)

Configuration

Config: `cs336-basics/configs/experiment/your_data.yaml` Slurm job: `scripts/train_job.slurm`

Parameter	Value
Model	GPT-2 small (124M params), 12 layers, 768 hidden dim, 12 heads
Hardware	2x NVIDIA RTX A4000 (16 GB each), Koa ece405 partition
Batch size per device	16

Parameter	Value
Gradient accumulation steps	8
Effective batch size	16 × 8 × 2 GPUs = 256 sequences/step
Tokens per step	256 × 512 = 131,072
Precision	bfloat16
torch.compile	Disabled (OOM on A4000)
Training steps	12,000
Eval interval	Every 1,000 steps
Total tokens seen	~1.57B (18% of 8.7B dataset)
Training time	13h 56m (Slurm job 11294442)
wandb	offline mode, synced post-hoc

Hardware adaptation note: The default training configuration (batch_size=128, torch.compile=True, 200K steps) was designed for the Together cluster's A100 GPUs (40–80 GB VRAM). On Koa's RTX A4000 GPUs (16 GB), we reduced batch size from 128 to 16 and disabled torch.compile to fit in VRAM, compensating with gradient accumulation (8 steps) to preserve the same effective batch size of 256 sequences per step. Training steps were reduced from 200K to 12,000 to fit within the 20-hour Slurm wall time limit.

Training command:

```
cd cs336-basics
uv run torchrun --standalone --nproc_per_node=2 scripts/train.py
        --config-name=experiment/your_data
```

Validation loss curve

Validation was measured on a 10M-token held-out split from our own filtered corpus (Paloma C4 100 domains benchmark was not available on the Koa cluster).

Step	Val Loss	Wall Time
1,000	4.032	1h 09m
2,000	3.683	2h 12m
3,000	3.507	3h 21m
4,000	3.379	4h 34m
5,000	3.273	5h 47m
6,000	3.202	6h 58m
7,000	3.101	8h 08m
8,000	3.030	9h 17m
9,000	2.982	10h 25m
10,000	2.927	11h 33m
11,000	2.883	12h 42m

Step	Val Loss	Wall Time
12,000	**2.856**	**13h 56m**

Best validation loss: 2.856 at step 12,000 (final step).

The loss decreased consistently throughout training with no signs of plateauing, suggesting further training would improve results. At the current rate of improvement (~0.03 per 1,000 steps), completing the full 200K steps could potentially reduce the loss to ~2.3–2.5, though diminishing returns are expected.

wandb run: https://wandb.ai/pavelbushuyeu-university-of-hawaii-system/ece405-data/runs/6n6ms27f

Model weights: https://huggingface.co/bushuyeu/gpt2-small-cc-filtered